INSTITUTO UNIVERSITÁRIO DE LISBOA

Projeto Redes Óticas

Estudo sobre Aspetos do Planeamento de uma Rede Óptica

Marcos Figueiredo Nº111159
Tiago Inácio Nº110890

Introdução

O objetivo principal deste trabalho é o planeamento da rede através do cálculo de caminhos ópticos e da atribuição de comprimentos de onda (RWA) para o tráfego definido na Figura 2 da proposta do projeto. Para o encaminhamento, utilizam-se algoritmos de caminho mais curto com critérios de desempate baseados no menor número de saltos e no balanceamento de carga entre as ligações. No que diz respeito à atribuição de recursos, são comparadas as técnicas de *First Fit* (seguindo a estratégia *shortest path first*) e a coloração de grafos.

Adicionalmente, o estudo foca-se no dimensionamento do hardware necessário, calculando o número de transponders em cada nó e em projetar a arquitetura interna do ROADM do Nó 1. Esta arquitetura vai seguir o modelo *Route & Select* com uma estrutura de Add/Drop CDC (Colorless, Directionless e Contentionless). Por fim, analisa-se o impacto da proteção dedicada de canal (1+1) na disponibilidade da rede e no uso de recursos óticos.

Ainda no seguimento do projeto vamos relatar os nossos resultados na parte mais prática do trabalho para o RWA 2: Algoritmo First Fit, Most Used e Random, bem como mostrar e explicar o simulador realizado para os obter.

Parte I

A-1)

Inicialmente foi pedido para calcularmos as ligações lógicas ordenadas, pela menor distância do caminho. Em caso de empate o critério adotado foi o menor número de saltos e se necessário, recorrer ainda à melhor distribuição de carga do caminho. Começámos por listar todas as ligações lógicas possíveis (malha total) entre os elementos do grafo que nos foi dado. Em seguida, eliminámos aqueles cuja matriz de tráfego não tinha tráfego direcionado.

Após chegarmos aos caminhos mais curtos (em termos de distância), e em caso de empate selecionarmos aqueles com menor número de saltos e com melhor balanceamento de carga, ordenámos as ligações pela menor distância.

Obtivémos então a seguinte tabela:

Ligação lógica	Distância [km]	Caminho físico
1-3	300	1-3
4-6	300	4-6
5-6	300	5-6
1-2	500	1-2
3-4	500	3-4
4-5	500	4-5
6-7	500	6-7
2-3	700	2-3
1-4	800	1-3-4
3-7	800	3-1-7
5-7	800	5-6-7
1-6	1000	1-7-6
3-5	1000	3-4-5
2-4	1200	2-3-4
1-5	1300	1-7-6-5

2-6	1500	2-1-7-6
2-5	1700	2-3-4-5

Tabela 1- Ligações lógicas com tráfego direcionado ordenadas

Durante a fase de cálculo das rotas, identificaram-se cenários de ambiguidade onde alguns caminhos apresentavam a mesma distância física (métrica principal) e o mesmo número de saltos (primeiro critério de desempate). Nestes casos, aplicou-se o critério de balanceamento de carga, que visa otimizar a distribuição de tráfego pela infraestrutura física e evitar a exaustão precoce de recursos em ligações específicas.

A análise para a tomada de decisão seguiu a seguinte metodologia :

- Para o par 2-6, onde as opções via Nó 1 ou via Nó 3 apresentavam métricas idênticas de distância e saltos, optou-se pelo caminho via Nó 1-7-6 por este atravessar ligações com menor carga acumulada no momento da análise, maximizando o equilíbrio operacional da rede.
- Para o par 1-5 onde as opções via Nó 3 ou via Nó 7 optou-se pelo caminho 2-1-7-6 pela exata razão de atravessar ligações com menos carga acumulada.

2)

Após obtermos a tabela do tráfego pelas ligações utilizamos a estratégia do First Fit em conjunto com SPF(*Shortest Path First)* para calcularmos o mínimo número de comprimentos de onda necessários para este sistema.

O First Fit consiste em atribuir o comprimento de onda com o menor índice disponível a cada pedido, sendo que este comprimento de onda deve ser utilizado em todo o caminho. Assim, para que um comprimento de onda seja elegível para um determinado caminho, este não pode estar a ser utilizado em nenhuma parte do mesmo.

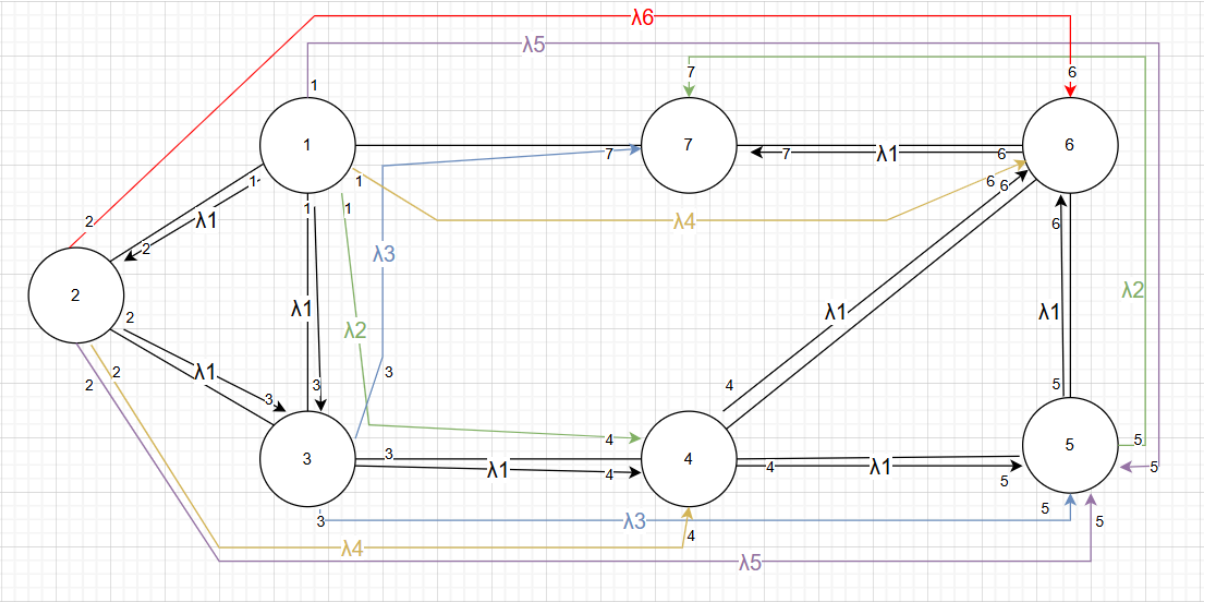

Figura 1 - A nossa topologia física com os diversos comprimentos de onda atribuídos aos vários caminhos óticos utilizando o algoritmo First Fit.

Pela figura, concluímos que necessitamos de pelo menos 6 comprimentos de onda diferentes para que a nossa rede funcione.

De seguida foi-nos proposto para fazermos a atribuição de comprimentos de onda, mas utilizando a estratégia da coloração de grafos. Para fazer a atribuição de comprimentos de onda com base na coloração de grafos, começámos por fazer com que os vértices do grafo fossem os caminhos óticos. Depois fizemos as ligações entre os nós (vértices) que tivessem alguma ligação física em comum.

Conseguimos assim o seguinte grafo:

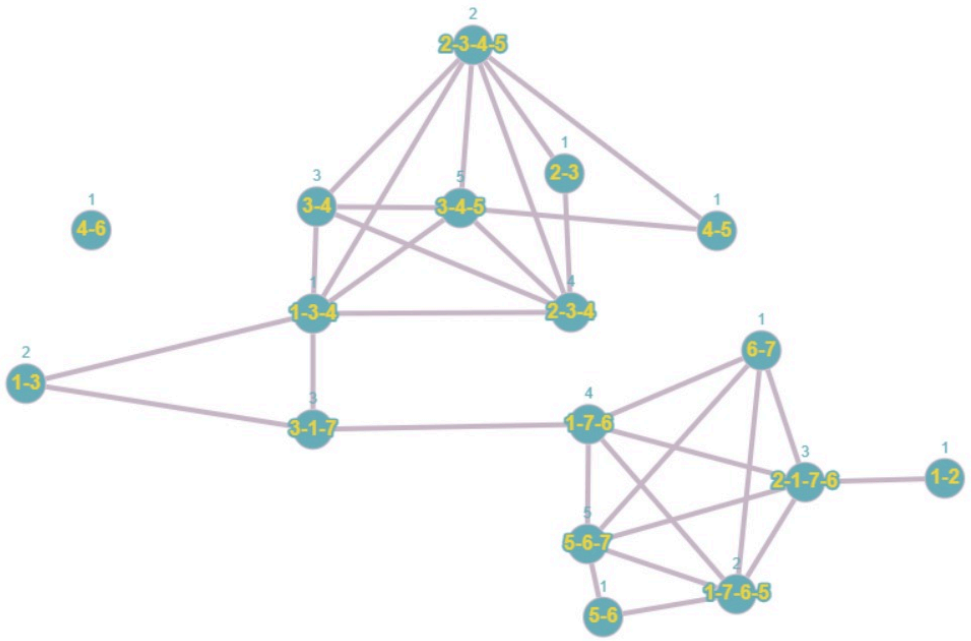

Figura 2 - Grafo sem cores atribuídas

Após termos o nosso grafo inicial construído, fizemos uma tabela com as seguintes colunas: vértice, grau e comprimento de onda.

Ordenámos os vértices do maior para o menor grau e em caso de empate considerámos o menor número de saltos e, caso o empate persistisse considerámos o menor valor numérico dos elementos do caminho. Assim com as linhas ordenadas preenchemos a coluna do comprimento de onda de cima para baixo, com a primeira cor disponível que não fosse utilizada por nenhum dos vértices adjacentes com cor já atribuída.

Obtivémos então a seguinte tabela:

Vértice	Grau	Comprimento de onda
1-3-4	6	λ1
2-3-4-5	6	λ2
1-7-6	5	λ1
2-3-4	5	λ3
3-4-5	5	λ4
5-6-7	5	λ2
1-7-6-5	5	λ3
2-1-7-6	5	λ4

3-4	4	λ5
6-7	4	λ5
3-1-7	3	λ2
1-3	2	λ3
2-3	2	λ1
4-5	2	λ1
5-6	2	λ1
1-2	1	λ1
4-6	0	λ1

Tabela 2 - Informações que permitem a coloração do grafo

Tendo então as cores atribuídas colorimos o nosso grafo, em que obviamente os números das cores correspondem aos números dos lambdas da tabela:

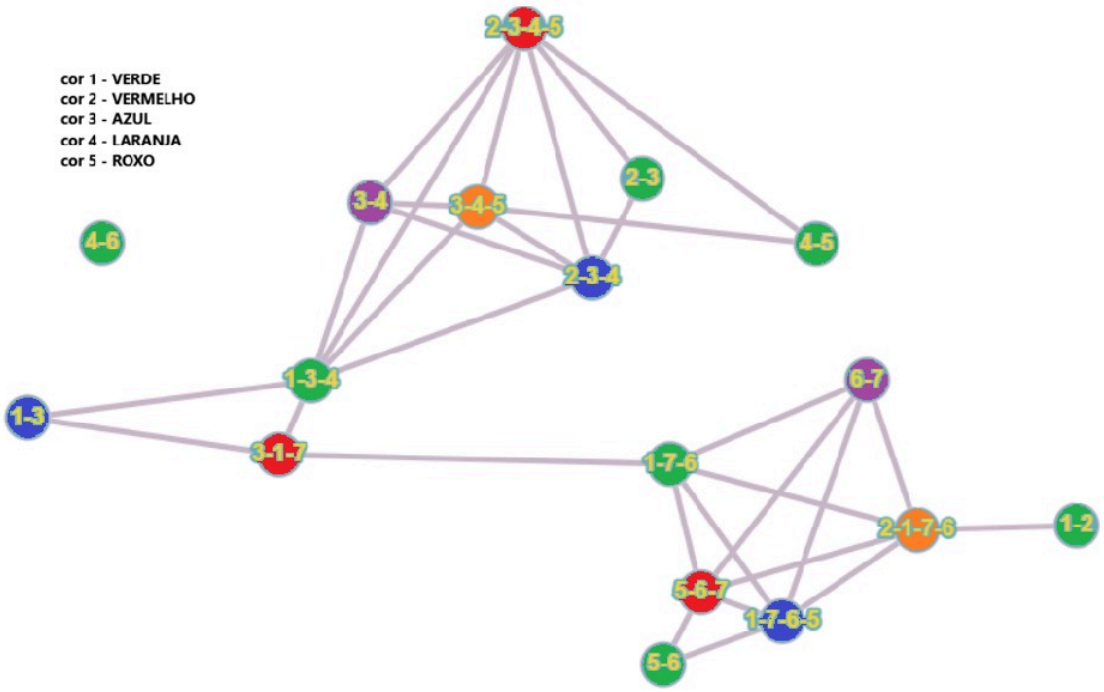

Figura 3- Grafo com as cores atribuídas

Podemos ver pela tabela e pela figura, que para o cenário da coloração de grafos o nosso número cromático do grafo é 5, ou seja no mínimo, precisamos de cinco cores para

colorir o grafo. Assim, precisamos de pelo menos 5 comprimentos de onda diferentes para a nossa rede.

Comparando com o algoritmo First Fit, o resultado obtido com a técnica de coloração de grafos apresenta uma ligeira melhoria, uma vez que necessitamos de menos um comprimento de onda para que a nossa rede funcione.

B)

Para o cálculo do número de transponders necessários por cada nó foi feito o cálculo através do número de OTU-k que cada nó suporta como sua origem. Este valor vai ser depois multiplicado por dois para obter o número final de transponders por nó devido ao tráfego ser bidirecional, ou seja, pode ser efetuado pelos dois sentidos.

Nó 1= 4+4+6+2+2+0=18, Nó 1= $18 \times 2 = 36$

Nó 2=4+0+3+2+2+2+0=13, Nó 2= $13 \times 2 = 26$

Nó 3=4+3+0+4+2+0+6=19, Nó 3= $19 \times 2 = 38$

Nó 4=6+2+4+0+3+5+0=20, Nó 4= $20 \times 2 = 40$

Nó 5=2+2+2+3+0+5+6=20, Nó 5= $20 \times 2 = 40$

Nó 6=2+2+0+5+5+0+8=22, Nó 6= $22 \times 2 = 44$

Nó 7=0+0+6+0+6+8+0=20, Nó 7= $20 \times 2 = 40$

$$\sum_{Nó\,1}^{Nó\,7} num\ transponders = 36 + 26 + 38 + 40 + 40 + 44 + 40 = 264$$

De seguida foi nos pedido para apresentar uma estrutura possível para a arquitectura do ROADM do nó 1 e, admitindo que se usa uma arquitectura Route & Select com uma estrutura de A/D CDC vamos ter um ROADM como o apresentado na figura abaixo :

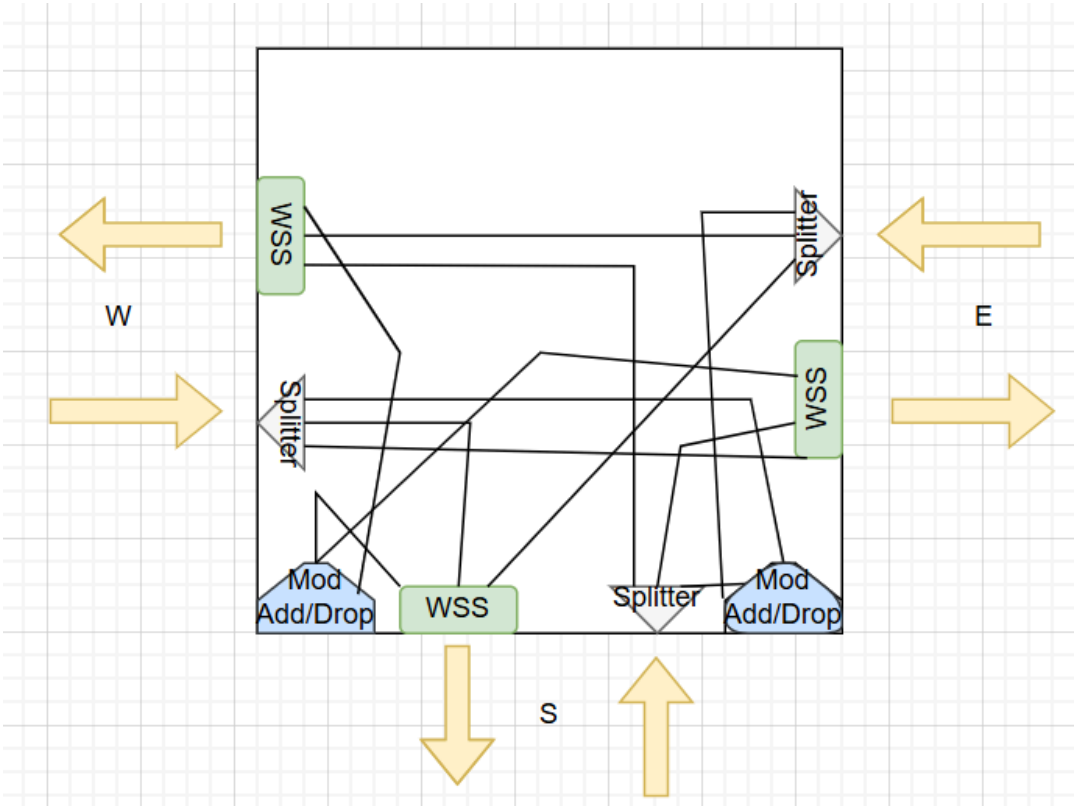

Figura 4- ROADM CDC com grau 3

A arquitetura ilustrada representa a configuração interna do Nó 1, a operar com Grau 3 (ligações para os nós 2, 3 e 7). Este nó utiliza uma estrutura CDC (Colorless, Directionless, Contentionless), essencial para a flexibilidade da rede ótica moderna. Para esta representação tivemos como base a arquitetura de um ROADM CDC de grau 4 e, foi depois alterado para grau 3 devido ao nó 1 só ter 3 ligações diretas e, por isso, não necessitar de ser de grau 4.

C)

Neste ponto vamos calcular os caminhos de serviço e protecção para a rede considerada, admitindo que se usa protecção dedicada de canal (1+1).

A proteção dedicada de canal (1+1) é uma estratégia de sobrevivência em redes óticas onde, para cada pedido de tráfego entre uma origem e um destino, são reservados dois caminhos distintos que operam em simultâneo. O Caminho de Serviço é a rota principal, selecionada através da topologia física para ser a mais eficiente, geralmente correspondendo ao caminho mais curto em termos de distância. Paralelamente, estabelece-se o Caminho de Proteção, uma rota alternativa que deve ser obrigatoriamente disjunta de nós e de ligações em

relação ao caminho principal. Esta disjunção garante que nenhuma falha única num cabo ou num nó intermédio possa interromper ambos os caminhos ao mesmo tempo.

Na prática, o sinal é transmitido permanentemente por ambas as rotas; se o recetor detetar uma falha no caminho de serviço, comuta instantaneamente para o sinal que já está a chegar pelo caminho de proteção, garantindo a continuidade do serviço sem perda de dados.

Par	Distância serviço(km)	Distância proteção(km)	Caminho Serviço	Caminho Proteção
(1,2)	500	1000	1-2	1-3-2
(1,3)	300	1200	1-3	1-2-3
(1,4)	800	1300	1-3-4	1-7-6-4
(1,5)	1300	1300	1-7-6-5	1-3-4-5
(1,6)	1000	1100	1-7-6	1-3-4-6
(2,3)	700	800	2-3	2-1-3
(2,4)	1200	1800	2-3-4	2-1-7-6-4
(2,5)	1700	1800	2-3-4-5	2-1-7-6-5
(2,6)	1500	1500	2-1-7-6	2-3-4-6
(3,4)	500	1600	3-4	3-1-7-6-4
(3,5)	1000	1600	3-4-5	3-1-7-6-5
(3,7)	800	1300	3-1-7	3-4-6-7
(4,5)	500	600	4-5	4-6-5
(4,6)	300	800	4-6	4-5-6
(5,6)	300	800	5-6	5-4-6
(5,7)	800	1800	5-6-7	5-4-3-1-7
(6,7)	500	1600	6-7	6-4-3-1-7

Tabela 3 - Ligações lógicas com as distâncias dos caminhos de serviço e proteção, e respectivos caminhos físicos.

Como próximo passo para voltar a atribuir comprimentos de onda vamos voltar a ter de ordenar os caminhos de forma ascendente para usarmos o First Fit com base no SPF(*Shortest Path First)* que decidimos voltar a usar para manter coerência com o ponto A. A respetiva tabela encontra-se de seguida.

Ligação lógica	Distância [km]	Caminho físico
1-3	300	1-3
4-6	300	4-6
5-6	300	5-6
1-2	500	1-2
3-4	500	3-4

4-5	500	4-5
6-7	500	6-7
(4-5)p	600	4-6-5
2-3	700	2-3
1-4	800	1-3-4
(2-3)p	800	2-1-3
3-7	800	3-1-7
(4-6)p	800	4-5-6
(5-6)p	800	5-4-6
5-7	800	5-6-7
1-6	1000	1-7-6
3-5	1000	3-4-5
(1-2)p	1000	1-3-2
(1,6)p	1100	1-3-4-6
2-4	1200	2-3-4
(1-3)p	1200	1-2-3
(1-4)p	1300	1-7-6-4
(1-5)p	1300	1-3-4-5
(3-7)p	1300	3-4-6-7
1-5	1300	1-7-6-5
2-6	1500	2-1-7-6
(2-6)p	1500	2-3-4-6
(3-4)p	1600	3-1-7-6-4
(3-5)p	1600	3-1-7-6-5
(6-7)p	1600	6-4-3-1-7
2-5	1700	2-3-4-5
(2-4)p	1800	2-1-7-6-4
(2-5)p	1800	2-1-7-6-5

(5-7)p	1800	5-4-3-1-7

Tabela 4 - Representação de todos os caminhos com tráfego, incluindo os de proteção

Nota: na tabela as ligações que correspondem a caminhos de proteção encontram-se identificados como(x-y)p, onde "p" marca que este é um caminho de proteção.

Decidimos seguir o SPF(*Shortest Path First*) para a atribuição de comprimentos de onda com base no First Fit. Este procedimento foi realizado teoricamente e, de seguida encontra-se a imagem que mostra os comprimentos de onda que passam em cada ligação entre os nós vizinhos.

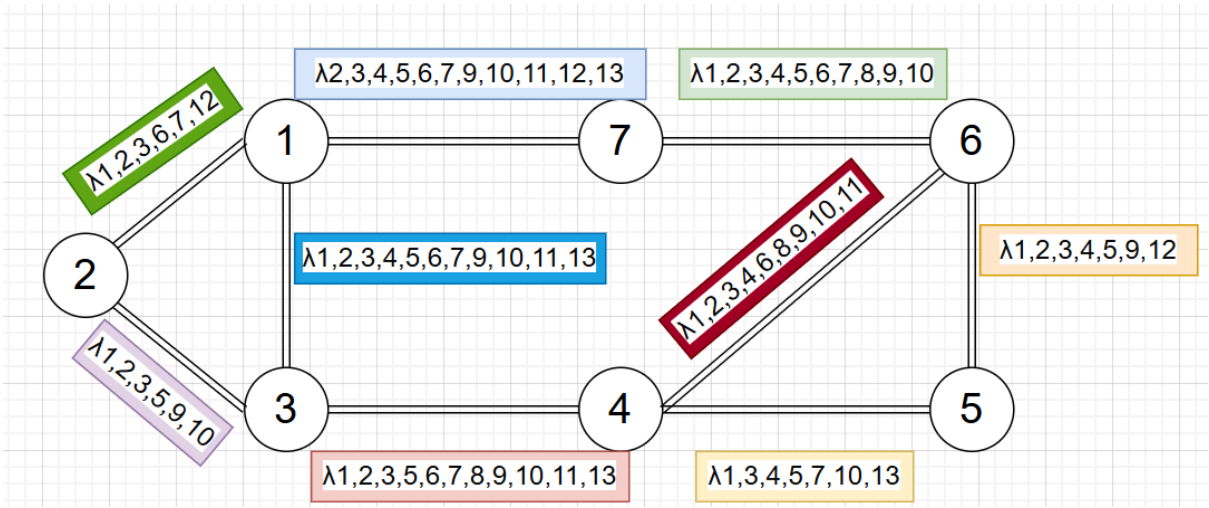

Figura 5- Atribuição de comprimentos usando First Fit e com proteção dedicada de canal(1+1)

Como podemos observar vão ser necessários 13 comprimentos de onda no mínimo para o nosso sistema, agora que temos implementada proteção dedicada de canal (1+1), este valor é mais do dobro do que seria sem a implementação dos caminhos de proteção e, apesar de necessitarmos de mais comprimentos de onda, o que vai ser mais dispendioso, a disponibilidade do nosso sistema vai aumentar significativamente como vamos poder observar no ponto que se segue.

D)

A disponibilidade define-se como a probabilidade de um elemento ou sistema estar operacional num determinado instante de tempo. Para quantificar este valor, utilizamos dois indicadores fundamentais: o MTBF (Mean Time Between Failures), que representa o tempo médio entre falhas consecutivas de um elemento; o MTTR (Mean Time To Repair): É o

tempo médio necessário para reparar uma falha e restaurar o serviço Para fibras óticas, o valor típico é de 24 horas.

A disponibilidade de um único elemento é calculada pela fórmula :

$$A = 1 - \frac{MTTR}{MTTR + MTBF}$$

Primeiro vamos calcular a disponibilidade no cenário apresentado no ponto A:

$$MTBF = (CC \times 365 \times 24) / \, Comprimento\ do\ cabo$$

Como é associado ao caminho mais longo temos Comprimento do cabo = 1700km

$$MTBF = \frac{500 \times 365 \times 24}{1700} = 2576,47h$$

Assumindo $MTTR = 24$h

$$Disponibilidade\ do\ cabo\ vai\ ser\ então = 1 - \frac{24}{24+2576,47} = 0,990771$$

A disponibilidade desta conexão vai ser então $A(cabo) \times A(nó)^{num\ de\ nós}$,

como a nossa ligação passa por 4 e assumindo $A(ROADM) = 0,99999$ vamos ter :

$$A\ da\ ligação = (0,99999)^4 \times (0,990771) = 0,99073 = 99,073\%$$

Para o caso do ponto C vamos ter :

$$MTBF = \frac{500 \times 365 \times 24}{1800} = 2433,33\text{h}$$

$$Disponibilidade\ do\ cabo\ vai\ ser\ então = 1 - \frac{24}{24+2433,33} = 0,990234$$

$$U\ cabo = 0,009766$$

$$U\ total = 0,009766 \times (0,99999)^5 = 0,009766$$

Como na situação do ponto C temos proteção, os dois percursos estão em paralelo e em caso de um não funcionar vamos ter o outro, logo a indisponibilidade combinada vai ser:

$$U(pontoA) \times U(pontoB) = 0,00927 \times 0,009766 = 0,00009053$$

Logo a disponibilidade total vai ser dada por :

$$A = 1 - U$$

$A = 0,99990947\ que\ corresponde\ a\ 99,990947\ \%$, valor bastante superior ao obtido para a disponibilidade com base no ponto A.

A proteção dedicada (1+1) melhora drasticamente a disponibilidade porque substitui uma configuração em série por uma estrutura em paralelo, onde o sinal é enviado simultaneamente por dois caminhos fisicamente disjuntos. Desta forma, a conexão deixa de estar vulnerável a um único corte de fibra, passando a falhar apenas se ocorrerem avarias em ambos os percursos ao mesmo tempo. Esta redundância permite que a indisponibilidade total seja o produto das indisponibilidades individuais, elevando o serviço de "2 noves" para "4/5 noves".

PARTE II

Para a implementação Matlab, o objetivo é encontrar o número mínimo de comprimentos de onda para as redes da figura 1 do enunciado do projeto e para a rede alemã. Para o primeiro caso assumimos a topologia lógica definida pela matriz de tráfego correspondente.

Para o segundo caso, assumimos uma topologia lógica em malha e admitimos também uma unidade de tráfego para todos os caminhos.

Para facilitar a implementação referente à rede alemã, numerámos as cidades da seguinte forma:

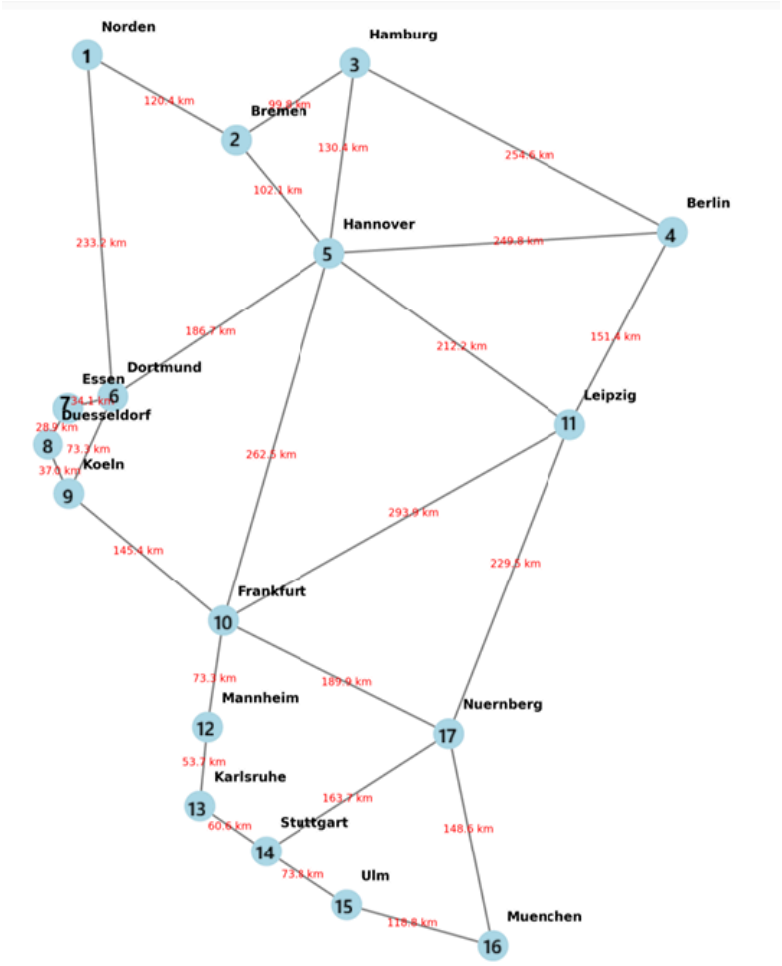

Figura 6- Rede alemã com a nossa numeração.

Número	Cidade
1	Norden
2	Bremen
3	Hamburg
4	Berlin
5	Hannover
6	Dortmund
7	Essen
8	Dusseldorf
9	Koeln
10	Frankfurt
11	Leipzig
12	Mannheim

13	Karlsruhe
14	Stuttgart
15	Ulm
16	Muenchen
17	Nuernberg

Tabela 5 - Numeração dada às cidades da rede alemã.

Código e resultados obtidos para a figura 1:

```
% 1. DEFINIÇÃO DA TOPOLOGIA
nodes = 7;
s = [1 1 2 3 3 4 5 6 7];
t = [2 7 3 1 4 5 6 4 6];
distancias = [500 500 700 300 500 500 300 300 500];
G = graph(s, t, distancias);

% 2. MATRIZ DE TRÁFEGO
T = [0 4 4 6 2 2 0;
    4 0 3 2 2 2 0;
    4 3 0 4 2 0 6;
    6 2 4 0 3 5 0;
    2 2 2 3 0 5 6;
    2 2 0 5 5 0 8;
    0 0 6 0 6 8 0];
% 3. ENCAMINHAMENTO COM PREFERÊNCIA (Balanceamento via Nó 7)
pedidos = [];
for u = 1:nodes
   for v = u+1:nodes
       if T(u,v) > 0
         % Encontrar o caminho padrão
           path = shortestpath(G, u, v);

           % 2. Lógica de Preferência para Balanceamento (Nó 7)
           % Se o caminho for entre nós que podem passar pelo topo (1,7,6)
             % tentamos forçar esse desvio para evitar o congestionamento do
3-4
           if (u == 1 && v == 5) || (u == 2 && v == 6)
```

```
                % Forçamos o caminho a passar pelo nó 7 para balancear a carga
                path_alt1 = [shortestpath(G, u, 7), shortestpath(G, 7, v)];
                 path = unique(path_alt1, 'stable'); % Remove duplicados do nó
7
            end
            % Mapeamento de arestas e distância
            edge_idx_list = [];
            d_total = 0;
            for k = 1:(length(path)-1)
                idx = findedge(G, path(k), path(k+1));
                edge_idx_list = [edge_idx_list, idx];
                d_total = d_total + G.Edges.Weight(idx);
            end

            p.u = u; p.v = v;
            p.edges = edge_idx_list;
            p.caminho_nos = path;
            p.dist = d_total;
            p.hops = length(edge_idx_list);
            pedidos = [pedidos; p];
        end
    end
end
% 4. ORDENAÇÃO SPF (Shortest Path First)
[~, idx_ord] = sortrows([[pedidos.dist]', [pedidos.hops]'], [1 2]);
pedidos_ordenados = pedidos(idx_ord);
% 5. SIMULAÇÃO RWA (Comparação de Métodos)
metodos = {'First Fit', 'Most Used', 'Random'};
for m = 1:length(metodos)
    metodo_atual = metodos{m};

    uso_espectro = zeros(numedges(G), 100);

    for i = 1:length(pedidos_ordenados)
        edges = pedidos_ordenados(i).edges;
        l_final = 0;

        switch metodo_atual
            case 'First Fit'
                % Menor índice disponível
                for L = 1:size(uso_espectro, 2)
```

```
                    if all(uso_espectro(edges, L) == 0), l_final = L; break;
end
                end

            case 'Most Used'
                % Prioriza lambdas com maior ocupação na rede
                utilizacao_global = sum(uso_espectro, 1);
                [~, ordem_uso] = sort(utilizacao_global, 'descend');
                for L = ordem_uso
                     if utilizacao_global(L) > 0 && all(uso_espectro(edges, L)
== 0)
                        l_final = L; break;
                    end
                end
                % Se nenhum usado servir, recorre ao First Fit
                if l_final == 0
                    for L = 1:size(uso_espectro, 2)
                            if all(uso_espectro(edges, L) == 0), l_final = L;
break; end
                    end
                end

            case 'Random'
                % Escolha aleatória entre os disponíveis
                candidatos = [];
                for L = 1:size(uso_espectro, 2)
                    if all(uso_espectro(edges, L) == 0)
                        candidatos = [candidatos, L];
                        if length(candidatos) >= 10, break; end % Limita
                    end
                end
                l_final = candidatos(randi(length(candidatos)));
        end

        % Ocupação do espectro
        uso_espectro(edges, l_final) = 1;
    end

    % Cálculo do resultado
    n_min = max(find(sum(uso_espectro, 1) > 0));
```

```
    fprintf('METODO: %-10s | N° MÍNIMO DE Comprimentos de Onda: %d\n',
metodo_atual, n_min);
end
```

O simulador inicia-se com a criação do modelo físico da rede através da função graph(s, t, distancias), que mapeia os 7 nós e os seus 9 elos bidirecionais, atribuindo a cada um a distância real a que se encontra do outro em quilómetros. Para processar a procura, fazemos dois ciclo for , onde "v = u+1:node", que percorre apenas o triângulo superior da matriz de tráfego "T", garantindo que cada ligação lógica entre dois nós seja tratada como um único par bidirecional, evitando assim a duplicação artificial de carga na rede.
Dentro deste ciclo, a função *shortestpath* aplica o algoritmo de Dijkstra para encontrar a rota que minimiza a distância total. Para otimizar o desempenho, foi inserida uma condição lógica para o balanceamento de carga (if (u == 1 && v == 5) || (u == 2 && v == 6)) que identifica fluxos críticos e força o seu desvio através do Nó 7. Este desvio manual evita o congestionamento do elo central 3-4, permitindo uma melhor distribuição dos sinais pela periferia da rede.

Antes de iniciar a atribuição de comprimentos de onda, o comando *sortrows* ordena a lista de pedidos de forma ascendente, baseando-se na distância física e no número de saltos. Esta estratégia de Shortest Path First (SPF) é vital para a eficiência do sistema, pois ao satisfazer primeiro os pedidos que ocupam menos recursos, criam-se mais oportunidades para a reutilização espacial de comprimentos de onda em zonas distintas da topologia.

No processo de atribuição , usamos a condição "all(uso_espectro(edges, L) == 0)" para verificar a continuidade de comprimento de onda, assegurando que o canal selecionado está livre em todo o percurso de ponta a ponta. O algoritmo First Fit seleciona sistematicamente o primeiro índice disponível, enquanto o Most Used utiliza o comando "sort(utilizacao_global, 'descend')" para priorizar canais que já estão em uso noutras partes da rede, tentando manter o espetro o mais compacto possível. Em contraste, o método Random utiliza a função *randi* para escolher um canal ao acaso, o que serve para demonstrar como uma má gestão do espetro pode levar à necessidade de mais recursos. Finalmente, fazemos "max(find(sum(uso_espectro, 1) > 0))", que identifica o comprimento de onda de índice mais elevado utilizado, fornecendo o resultado final apresentado de seguida..

```
METODO: First Fit   | Nº MÍNIMO DE Comprimentos de Onda: 6
METODO: Most Used   | Nº MÍNIMO DE Comprimentos de Onda: 6
METODO: Random      | Nº MÍNIMO DE Comprimentos de Onda: 12
```

Figura 7- Resultados obtidos para a topologia da figura 1 do enunciado

Os algoritmos First Fit e Most Used revelaram-se os mais eficientes, conseguindo satisfazer todos os caminhos com apenas 6 comprimentos de onda. Esta convergência de resultados indica que, para esta topologia específica, tanto a compactação linear do First Fit como a priorização de canais populares do Most Used conseguem maximizar a reutilização de comprimentos de onda nos elos menos congestionados.

Em contraste, o método Random apresentou um desempenho significativamente inferior, exigindo 12 comprimentos de onda para a mesma carga de tráfego, o que representa um aumento muito elevado no consumo de recursos. Este comportamento justifica-se pela elevada fragmentação que a seleção aleatória provoca, dificultando o encontro de canais livres contínuos para pedidos que atravessam múltiplos saltos.

Código e resultados obtidos para a rede alemã:

```
%grafo rede alema
s = [1 1 2 2 3 3 4 4 5 5 5 6 6 7 8 9 10 10 10 11 12 13 14 14 15 16];
t = [2 6 3 5 4 5 5 11 6 10 11 7 9 8 9 10 11 12 17 17 13 14 17 15 16 17];
w = [120.4 233.2 99.8 102.1 254.6 130.4 249.8 151.4 186.7 262.5 212.2 34.1
73.3  28.9  37  145.4  293.9  73.3  189.9  229.5  53.7  60.6  163.7  73.8  116.8
148.6];
G = graph(s, t, w);
%geração dos pedidos da rede alemã numa topologia lógica em malha
N=17;
requests = [];
for i = 1:N
   for j = i+1:N
       requests = [requests; i j];
   end
end
numPedidos=size(requests,1);
numNodes = numnodes(G);
caminhoDistancia = cell(numPedidos, 2);
for k = 1:numPedidos
   origem  = requests(k, 1);
   destino = requests(k, 2);

   [P, d] = shortestpath(G, origem, destino, 'Method', 'positive');

   caminhoDistancia{k, 1} = P;
   caminhoDistancia{k, 2} = d;
end
distancias = zeros(numPedidos,1);
for i = 1:numPedidos
   distancias(i) = caminhoDistancia{i,2};
end
[valoresOrdenados, indices] = sort(distancias);
caminhoDistanciaOrdenado = cell(numPedidos,2);
for i = 1:numPedidos
   caminhoDistanciaOrdenado{i,1} = caminhoDistancia{indices(i),1};
   caminhoDistanciaOrdenado{i,2} = caminhoDistancia{indices(i),2};
end

caminhoDistancia = caminhoDistanciaOrdenado;
```

```
%FIRST FIT
numWavelengthsFF = 1;
wavelengthUsageFF = zeros(1, numNodes, numNodes);
for r = 1:numPedidos
   path = caminhoDistanciaOrdenado{r,1};
   atribuido = 0;
   for w = 1:numWavelengthsFF
       livre = 1;
       for k = 1:length(path)-1
           if wavelengthUsageFF(w, path(k), path(k+1)) == 1
               livre = 0;
           end
       end
       if livre == 1
           for k = 1:length(path)-1
               wavelengthUsageFF(w, path(k), path(k+1)) = 1;
               wavelengthUsageFF(w, path(k+1), path(k)) = 1;
           end
           atribuido = 1;
           break;
       end
   end
   if atribuido == 0
       numWavelengthsFF = numWavelengthsFF + 1;
       for k = 1:length(path)-1
           wavelengthUsageFF(numWavelengthsFF, path(k), path(k+1)) = 1;
           wavelengthUsageFF(numWavelengthsFF, path(k+1), path(k)) = 1;
       end
   end
end
fprintf('Para o cenário da rede alema teremos %d comprimentos de onda com o
first fit\n', numWavelengthsFF);
%MOST USED
numWavelengthsMU = 1;
wavelengthUsageMU = zeros(1, numNodes, numNodes);
for r = 1:numPedidos
   path = caminhoDistanciaOrdenado{r,1};
   atribuido = 0;
     uso = zeros(numWavelengthsMU,1);
   for w = 1:numWavelengthsMU
       soma = 0;
```

```
        for i = 1:numNodes
            for j = 1:numNodes
                soma = soma + wavelengthUsageMU(w,i,j);
            end
        end
        uso(w) = soma;
    end
      while atribuido == 0
        maiorUso = -1;
        melhorW = 0;
        for w = 1:numWavelengthsMU
            if uso(w) > maiorUso
                livre = 1;
                for k = 1:length(path)-1
                    if wavelengthUsageMU(w, path(k), path(k+1)) == 1
                        livre = 0;
                    end
                end
                if livre == 1
                    maiorUso = uso(w);
                    melhorW = w;
                end
            end
        end
        if melhorW ~= 0
            for k = 1:length(path)-1
                wavelengthUsageMU(melhorW, path(k), path(k+1)) = 1;
                wavelengthUsageMU(melhorW, path(k+1), path(k)) = 1;
            end
            atribuido = 1;
        else
            numWavelengthsMU = numWavelengthsMU + 1;
            wavelengthUsageMU(numWavelengthsMU, :, :) = 0;
            for k = 1:length(path)-1
                wavelengthUsageMU(numWavelengthsMU, path(k), path(k+1)) = 1;
                wavelengthUsageMU(numWavelengthsMU, path(k+1), path(k)) = 1;
            end
            atribuido = 1;
        end
    end
end
```

```
fprintf('Para o cenário da rede alema teremos %d comprimentos de onda com o
most used\n', numWavelengthsMU);
%RANDOM
numWavelengthsR = 100;
WLused=[];
wavelengthUsageR = zeros(numWavelengthsR, numNodes, numNodes);
for r = 1:numPedidos
   path = caminhoDistanciaOrdenado{r,1};
   livres = [];

   for w = 1:numWavelengthsR
       livre = 1;
       for k = 1:length(path)-1
           if wavelengthUsageR(w, path(k), path(k+1)) == 1
               livre = 0;
           end
       end
       if livre == 1
           livres = [livres w];
       end
   end


   if length(livres) > 0
       indiceRandom = randi(length(livres));
       wEscolhido = livres(indiceRandom);
       if ~ismember(wEscolhido, WLused)
           WLused = [WLused wEscolhido];
       end
       for k = 1:length(path)-1
           wavelengthUsageR(wEscolhido, path(k), path(k+1)) = 1;
           wavelengthUsageR(wEscolhido, path(k+1), path(k)) = 1;
       end
   else
       numWavelengthsR = numWavelengthsR + 1;
       WLused=[WLused numWavelengthsR];
       for k = 1:length(path)-1
           wavelengthUsageR(numWavelengthsR, path(k), path(k+1)) = 1;
           wavelengthUsageR(numWavelengthsR, path(k+1), path(k)) = 1;
       end
   end
```

```
end
fprintf('Para o cenário da rede alema teremos %d comprimentos de onda com o
RANDOM\n', length(WLused));
```

A rede alemã foi implementada em MATLAB como um grafo ponderado não direcionado, onde os nós representam os nós óticos da rede e as ligações físicas são representadas por arestas ponderadas pelas respetivas distâncias. Considerando uma topologia lógica em malha completa, gerámos pedidos de tráfego unitário entre todos os pares distintos de nós da rede, resultando num conjunto total de pedidos correspondente a todas as combinações origem–destino possíveis (sem repetições de caminhos).

Para cada pedido, foi calculado o caminho mais curto entre a origem e o destino, utilizando o critério da menor distância física. O caminho obtido e a respetiva distância total foram armazenados para posterior processamento. De modo a melhorar a eficiência do processo de atribuição de comprimentos de onda, os pedidos foram ordenados por ordem crescente da distância do respetivo caminho, aplicando-se o critério Shortest Path First, que dá prioridade aos pedidos que consomem menos recursos da rede.

A atribuição de comprimentos de onda foi realizada garantindo a continuidade do comprimento de onda ao longo de todo o caminho e evitando conflitos nos enlaces da rede. Para esse efeito, foi utilizada uma matriz tridimensional que regista a ocupação de cada comprimento de onda em cada enlace da rede. Com base nesta estrutura, implementámos e avaliámos três algoritmos de atribuição de comprimentos de onda: First Fit, Most Used e Random.

No algoritmo First Fit, para cada pedido, os comprimentos de onda disponíveis são analisados sequencialmente, sendo atribuído o primeiro comprimento de onda que se encontra livre em todos os enlaces do caminho. Caso nenhum comprimento de onda existente satisfaça esta condição, é criado um novo comprimento de onda, que é imediatamente atribuído ao pedido. Este algoritmo caracteriza-se pela sua simplicidade e baixa complexidade computacional, servindo como referência para comparação com os restantes métodos.

No algoritmo Most Used, sempre que existem vários comprimentos de onda disponíveis para um determinado pedido, é selecionado aquele que apresenta maior ocupação global na rede, desde que esteja livre ao longo de todo o caminho considerado. Esta abordagem promove a concentração de tráfego em comprimentos de onda já utilizados, reduzindo a fragmentação espectral e, em muitos casos, o número total de comprimentos de onda necessários. Tal como nos restantes algoritmos, sempre que nenhum comprimento de onda existente esteja disponível, é criado um novo.

Por fim, no algoritmo Random, quando existem vários comprimentos de onda livres para um pedido, a escolha do comprimento de onda é efetuada de forma aleatória. Convém notar que de forma a que possa existir aleatoriedade desde a primeira escolha, decidimos começar com 100 comprimentos de onda disponíveis, ao contrário dos dois algoritmos anteriores, em que pelas características dos mesmos bastava apenas um. No entanto, tal como nos cenários anteriores, caso não exista qualquer comprimento de onda disponível, é criado um novo comprimento de onda para acomodar o pedido. Este algoritmo não segue qualquer critério de otimização e é utilizado essencialmente como termo de comparação, permitindo avaliar o impacto de decisões não determinísticas na utilização dos recursos da rede.

Os resultados desta implementação foram os seguintes:

```
Para o cenário da rede alema teremos 41 comprimentos de onda com o first fit
Para o cenário da rede alema teremos 41 comprimentos de onda com o most used
Para o cenário da rede alema teremos 84 comprimentos de onda com o RANDOM
```

Figura 8 - Resultados obtidos para a rede alemã.

Na sua devida proporção os resultados obtidos para a rede alemã foram semelhantes aos do primeiro cenário (rede da figura 1 do enunciado): O first fit e o most used obtiveram o mesmo resultado, sendo a lógica que explica esta igualdade igual ao que anteriormente foi explicado para o primeiro cenário. Também o Random teve um desempenho semelhante, já que necessitou de aproximadamente o dobro dos comprimentos de onda em relação ao first fit e most used (84 vs 41), tal como se verificou para o primeiro cenário (12 vs 6).

Conclusão

O presente projeto permitiu a análise e implementação de um sistema de planeamento de redes óticas, focado na resolução do problema de RWA (Routing and Wavelength Assignment). Através da utilização de diferentes estratégias conseguimos atribuir comprimentos de onda ao nosso sistema tanto teoricamente, como através de um simulador em *MATLAB*.

Relativamente à atribuição de comprimentos de onda, a comparação entre os algoritmos First Fit, Most Used e Random evidenciou a importância de uma gestão estruturada. Os métodos First Fit e Most Used demonstraram uma elevada eficiência na compactação de canais, permitindo satisfazer toda a matriz com apenas 6 comprimentos de onda para o cenário da figura 1 do enunciado. Em contrapartida, o método Random apresentou uma maior fragmentação, confirmando que a seleção arbitrária de recursos prejudica a escalabilidade da rede e aumenta o risco de bloqueio de futuros pedidos.

Em suma, o trabalho foi muito importante para a nossa sensibilização na atribuição de comprimentos de onda em cenários mais realistas, bem como o impacto que a utilização de caminhos de proteção têm na disponibilidade de um sistema. Para além disso, ainda identificamos a quantidade de transponders necessários para um certo cenário que também contribuiu para ganharmos mais conhecimento nesta área.